

Control Robot Left and Right Translation

Control Robot Left and Right Translation

- 1 Experiment Objective
- 2 Experiment Preparation
- 3 Experiment Procedure
- 4 Experiment Summary

1 Experiment Objective

This course focuses on learning how to control the robot's left and right translation functions.

2 Experiment Preparation

This course involves the following functions from the Muto hexapod robot's Python library:

left(step): Translate left. `step` is the stride width. A larger effective value results in a wider stride. The value range for `step` is [10, 25].

right(step): Translate right. `step` is the stride width. A larger effective value results in a wider stride. The value range for `step` is [10, 25].

3 Experiment Procedure

Open the JupyterLab client and navigate to the code path:

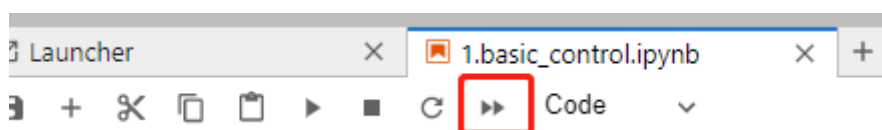
```
muto/Samples/Control/5.1left_right.ipynb
```

By default, `g_ENABLE_CHINESE` is `False`. If you need to display Chinese, please set `g_ENABLE_CHINESE=True`.

```
# Chinese switch. The default value is English
g_ENABLE_CHINESE = False

Name_widgets = {
    'Stop': ("Stop", "Stop"),
    'Left': ("Left", "Translate Left"),
    'Right': ("Right", "Translate Right"),
    'End': ("End", "Finish")
}
```

Click 'Run All Cells', then scroll to the bottom to see the generated controls.



Click the different buttons to perform their corresponding functions.

Step: 25
 step: 25
 Speed: 3

Left Right Stop End

Each time a button is clicked, the corresponding function is executed. The button event handling is as follows:

```
# Key press event processing
def on_button_clicked(b):
    ALL_Uncheck()
    b.icon = 'check'
    with output:
        print("Button clicked:", b.description)
    if b.description == Name_widgets['Stop'][g_ENABLE_CHINESE]:
        g_bot.stop()
        b.icon = 'uncheck'
    elif b.description == Name_widgets['Left'][g_ENABLE_CHINESE]:
        g_bot.left(g_step)
    elif b.description == Name_widgets['Right'][g_ENABLE_CHINESE]:
        g_bot.right(g_step)
```

The `g_step` variable is controlled by the Step slider. When the slider's value changes, it detects which button has been pressed and immediately updates the `g_step` value for the robot.

```
def on_slider_step(value):
    global g_step
    g_step = value
    if button_move_left.icon == 'check':
        g_bot.left(g_step)
    elif button_move_right.icon == 'check':
        g_bot.right(g_step)
    print("step:", value)
```

The robot's movement speed is controlled by the Speed slider. When the slider's value changes, the robot's movement speed is updated immediately.

```
def on_slider_speed(value):
    global g_bot
    g_bot.speed(value)
    print("speed:", value)
```

4 Experiment Summary

In this experiment, JupyterLab controls are used to manage the hexapod robot's speed and left/right translation functions. For example, set Step to 20 and Speed to 2, then click the Left button. The hexapod robot will translate left, the Left button will be checked, and the command pressed will be displayed below the controls. At this point, changing the values of the Step or

Speed sliders will change the robot's stride width and speed.

When you need to stop, click the Stop button.

Step: 25
step: 25

Speed: 3

Left Right Stop End

To exit the program, press the 'End' button.